Relazione Tecnica

Challenge-AI

Sistema di Analisi della Pulizia delle Vetrate tramite Intelligenza Artificiale

Realizzato da Chiavaroli Marco, Chindamo Andrea, Serra Benedetta, Zamboni Piero, Zini Arianna

Repository	mchiavaroli/Challenge-AI
Tecnologie principali	Next.js 16 · Python · FastAPI · EfficientNet-B0 · Supabase
Data relazione	Aprile 2026

Questa relazione descrive il progetto Challenge-AI: un'applicazione web che consente di gestire strutture edilizie (palazzi) e le loro vetrate, integrando un motore di intelligenza artificiale per valutare il livello di pulizia delle superfici vetrate tramite analisi di immagini fotografiche.

La relazione è organizzata in due parti: la **Parte 1** affronta l'analisi funzionale, i requisiti e lo scopo del progetto; la **Parte 2** descrive l'architettura tecnica e i dettagli implementativi della soluzione.

1. Analisi Funzionale e Requisiti

1.1 Contesto e Scopo del Progetto

Challenge-AI nasce con l'obiettivo di automatizzare il monitoraggio delle condizioni di pulizia delle vetrate di strutture edilizie (edifici residenziali, uffici, pannelli solari, facciate in vetro). La pulizia delle superfici vetrate è un fattore rilevante sia per ragioni estetiche sia funzionali: nel caso dei pannelli fotovoltaici, per esempio, uno strato di polvere riduce significativamente la resa energetica.

Il progetto propone una soluzione digitale che consente agli operatori di:

- Catalogare le strutture gestite (palazzi) e le relative superfici vetrate.
- Caricare fotografie delle vetrate direttamente dalla piattaforma web.
- Ottenere una valutazione automatica del livello di pulizia, espressa su scala 0–100, tramite un modello di deep learning.
- Visualizzare una classifica comparativa tra le strutture gestite, così da prioritizzare gli interventi di pulizia.

1.2 Stakeholder e Profilo Utente

Il sistema è progettato per essere utilizzato da facility manager, responsabili della manutenzione di edifici commerciali o residenziali, e da operatori del settore energia (gestori di impianti fotovoltaici). L'utente tipo non richiede competenze tecniche specifiche: l'interfaccia è pensata per essere intuitiva e orientata all'azione.

Gli stakeholder principali identificati sono:

- Operatori di manutenzione: inseriscono nuove strutture, caricano fotografie, richiedono valutazioni di pulizia.
- Responsabili di struttura: consultano la classifica comparativa per pianificare gli interventi.
- Team tecnico: gestisce il dataset di training e l'aggiornamento del modello AI.

1.3 Requisiti Funzionali

I requisiti funzionali del sistema sono stati organizzati per macro-area:

Gestione Strutture (Palazzi)

Questa sezione descrive le funzionalità dedicate alla gestione delle strutture (palazzi) all'interno del sistema. Include le operazioni di creazione, visualizzazione, modifica ed eliminazione dei palazzi, che rappresentano l'entità principale su cui si basano le altre funzionalità applicative.

ID	Descrizione
RF-01	Creazione di un nuovo palazzo con nome, descrizione e indirizzo.
RF-02	Visualizzazione elenco palazzi con contatore delle vetrate associate.
RF-03	Eliminazione di un palazzo (con cancellazione a cascata delle vetrate).
RF-04	Modifica dei dati di un palazzo esistente.

Gestione Vetrate

In questa sezione sono definiti i requisiti funzionali relativi alla gestione delle vetrate associate ai palazzi. Le funzionalità comprendono il caricamento, la visualizzazione e l'eliminazione delle immagini, che costituiscono il dataset utilizzato per le analisi successive.

ID	Descrizione
RF-05	Caricamento di immagini di vetrate associate a uno specifico palazzo.
RF-06	Visualizzazione della galleria di vetrate di un palazzo.
RF-07	Visualizzazione a schermo intero di una singola vetrata.
RF-08	Eliminazione di una vetrata.

Analisi AI della Pulizia

Questa sezione raccoglie le funzionalità legate all'analisi automatica del livello di pulizia dei palazzi tramite modelli di intelligenza artificiale. Il sistema elabora le immagini delle vetrate per produrre valutazioni quantitative e qualitative, oltre a fornire strumenti di confronto tra le diverse strutture.

ID	Descrizione
RF-09	Calcolo automatico del livello di pulizia medio di un palazzo a partire dalle immagini delle vetrate.
RF-10	Visualizzazione del risultato con punteggio numerico (0–100) e label qualitativa (Critico/Da migliorare/Buono/Eccellente).
RF-11	Generazione di una classifica comparativa della pulizia di tutti i palazzi gestiti.
RF-12	Segnalazione all'utente quando il modello AI non è ancora stato addestrato (risultato simulato).

1.4 Requisiti Non Funzionali

I requisiti non funzionali descrivono le caratteristiche qualitative del sistema, come usabilità, performance, scalabilità, affidabilità e sicurezza. Questi aspetti definiscono i vincoli e gli standard che il sistema deve rispettare per garantire un'esperienza utente efficace e un funzionamento robusto.

ID	Area	Descrizione
RNF-01	Usabilità	Interfaccia web responsiva, accessibile da desktop e dispositivi mobili.
RNF-02	Performance	Il calcolo del punteggio di pulizia deve completarsi in tempo ragionevole anche con più immagini.
RNF-03	Scalabilità	Architettura disaccoppiata (frontend + backend + ML service) per consentire scaling indipendente dei componenti.
RNF-04	Affidabilità	Il sistema deve gestire gracefully l'assenza del modello AI, restituendo un risultato simulato con avviso all'utente.
RNF-05	Sicurezza	Gestione sicura dei file caricati tramite Supabase Storage; comunicazione tra i layer tramite API REST.

1.5 Casi d'Uso Principali

Questa sezione presenta i principali scenari di utilizzo del sistema dal punto di vista dell'utente e del team tecnico. I casi d'uso descrivono le interazioni fondamentali con la piattaforma, evidenziando i flussi operativi più rilevanti.

ID	Caso d'Uso	Descrizione
UC-01	Creazione palazzo	L'utente inserisce nome, indirizzo e descrizione di una nuova struttura. Il sistema la salva e la mostra in lista.
UC-02	Caricamento vetrate	L'utente seleziona un palazzo e carica una o più fotografie di vetrate. Il sistema le salva su storage cloud.
UC-03	Calcolo pulizia palazzo	L'utente richiede l'analisi di un palazzo. Il sistema interroga il modello AI con le URL delle immagini e mostra il punteggio medio.
UC-04	Classifica pulizia	L'utente apre la classifica globale. Il sistema calcola il punteggio per ogni palazzo e li ordina dal più pulito al meno pulito.
UC-05	Training del modello	Il team tecnico esegue i script Python per etichettare le immagini e addestrare il modello; il modello aggiornato viene ricaricato dall'API.

2. Architettura Tecnica e Implementazione

2.1 Panoramica dell'Architettura

Il sistema è composto da tre layer principali, progettati secondo un'architettura a microservizi disaccoppiata che consente lo sviluppo, il deployment e lo scaling indipendente di ciascun componente:

Layer	Tecnologia	Responsabilità
Frontend	Next.js 16 (React 19)	Interfaccia utente + API Routes (Backend-for-Frontend)
Database & Storage	Memoria locale temporanea	Persistenza dati limitata alla singola esecuzione, sviluppata come soluzioni di testing
ML Service	Python / FastAPI	Inferenza del modello EfficientNet-B0 per predizione pulizia

Il flusso end-to-end è il seguente: l'utente interagisce con il frontend Next.js, che estrae localmente i dati (è possibile attivare una soluzione di persistenza dei dati su una DB supbase in cloud) e con il servizio Python per l'analisi AI. Il servizio ML riceve le URL delle immagini, le scarica, esegue l'inferenza e restituisce i punteggi, visualizzati in seguito a frontend.

2.2 Frontend — Next.js 16

Il frontend è realizzato con Next.js 16 in modalità App Router, con componenti React 19 scritti in TypeScript. L'interfaccia utilizza Tailwind CSS 4 per lo styling e i componenti shadcn/ui (basati su Radix UI) per elementi come dialog, card, button e progress bar.

I componenti principali dell'interfaccia sono:

Componente	Descrizione
page.tsx	Pagina principale: lista di palazzi con opzioni di creazione ed eliminazione.
PalazzoDetail	Vista di dettaglio di un palazzo: galleria vetrate, calcolo pulizia, risultato AI.
PalazzoCard	Card riepilogativa di un palazzo con contatore vetrate.
VetrataCard	Card di una singola vetrata con anteprima immagine e azioni.
UploadVetrataDialog	Dialog modale per il caricamento di nuove immagini.
CleanlinessRankingDialog	Dialog con classifica comparativa della pulizia di tutte le strutture.
ImageViewer	Visualizzatore a schermo intero di un'immagine vetrata.

Il data fetching è gestito tramite SWR per le chiamate GET (con caching e revalidazione automatica) e fetch nativo per le operazioni POST/DELETE.

2.3 API Routes — Backend-for-Frontend

Next.js espone API Routes che fungono da Backend-for-Frontend. Le route principali sono:

Endpoint	Descrizione
GET /api/palazzi	Lista tutti i palazzi con contatore vetrate.
POST /api/palazzi	Crea un nuovo palazzo.
GET /api/palazzi/[id]	Dettaglio di un singolo palazzo.
PATCH /api/palazzi/[id]	Aggiorna i dati di un palazzo.
DELETE /api/palazzi/[id]	Elimina un palazzo (cascade sulle vetrate).
POST /api/palazzi/[id]/cleanliness	Chiama il servizio ML e restituisce il punteggio medio di pulizia.
POST /api/palazzi/cleanliness	Calcola il punteggio per tutti i palazzi (classifica).
GET/POST/DELETE /api/vetrate/[id]	CRUD per le singole vetrate.
POST /api/upload	Caricamento di un file immagine su Supabase Storage.

2.4 Persistenza dei dati

La persistenza è gestita localmente, ma la soluzione permette anche una gestione tramite DB Supbase. Lo schema del database è composto da due tabelle principali:

Tabella palazzi

Colonna	Tipo	Note
id	UUID	Chiave primaria auto-generata (gen_random_uuid()).
name	TEXT NOT NULL	Nome della struttura.
description	TEXT	Descrizione opzionale.
address	TEXT	Indirizzo opzionale.
created_at	TIMESTAMPTZ	Timestamp di creazione (default: NOW()).
updated_at	TIMESTAMPTZ	Timestamp di ultima modifica.

Tabella vetrate

Colonna	Tipo	Note
id	UUID	Chiave primaria auto-generata.
palazzo_id	UUID FK	Riferimento a palazzi(id) con ON DELETE CASCADE.
name	TEXT NOT NULL	Nome/etichetta della vetrata.
description	TEXT	Descrizione opzionale.
image_url	TEXT NOT NULL	URL pubblica dell'immagine su Supabase Storage.
created_at	TIMESTAMPTZ	Timestamp di creazione.

È presente un indice su vetrate.palazzo_id per ottimizzare le query di selezione delle vetrate per palazzo.

2.5 Pipeline di Machine Learning

Il cuore innovativo del progetto è la pipeline di machine learning, composta da tre fasi distinte: etichettatura automatica del dataset, addestramento del modello e servizio di inferenza in produzione. Si utilizza EfficientNet-80, una rete neurale convoluzionale (CNN) progetta per analizzare immagini in modo efficiente. Addestrare una rete neurale da zero richiederebbe milioni di immagini. Per evitarlo, il backbone viene pre-caricato con pesi addestrati.

Fase 1 — Etichettatura Automatica (label_images.py)

Prima dell'addestramento, ogni immagine del dataset viene analizzata con un algoritmo di computer vision tradizionale che combina cinque indicatori visivi per stimare un punteggio di pulizia (0–100):

Indicatore	Peso	Significato
Saturazione colore	30%	Vetro pulito = colori intensi (blu profondo); sporco = colori sbiaditi e grigi.
Pixel luminosi/bianchi	20%	Polvere e sporco appaiono come macchie bianche brillanti.
Dominanza del blu	20%	Pannello pulito riflette un blu più marcato.
Nitidezza dei bordi	15%	Vetro pulito mostra linee nette; sporco rende l'immagine più sfocata.
Foschia grigia	15%	Zone grigiastre a bassa saturazione indicano sporco.

Il punteggio finale è la somma ponderata degli indicatori. È stata effettuata anche una verifica manuale delle etichette per garantire la qualità del dataset.

Fase 2 — Addestramento del Modello (train.py + model.py)

Il modello adottato è **EfficientNet-B0**, una rete neurale convoluzionale (CNN) nota per il suo eccellente trade-off tra accuratezza ed efficienza computazionale. L'architettura è composta da:

- Backbone (feature extractor): EfficientNet-B0 pre-addestrato su ImageNet (transfer learning). Estrae caratteristiche visive come bordi, texture e colori.
- Classifier head: Dropout(0.2) → Linear(1280 → 1) → Sigmoid(). Produce un valore scalare in [0, 1], scalato a [0, 100].

L'approccio di transfer learning con fine-tuning completo dell'intera rete (backbone + head) consente di partire da una base già esperta nel riconoscimento di forme e texture generiche e adattarla al dominio specifico delle vetrate.

I principali parametri di addestramento sono:

Parametro	Valore	Note
Epoche	20	Numero di passaggi completi sul dataset.
Batch size	16	Immagini processate per ogni step di aggiornamento.
Loss function	MSE	Errore quadratico medio (regressione).

Il training è diviso in epoche; per ognuna di essa il modello esegue i seguenti passi:

1. **Training**: per ogni gruppo di 16 immagini (valore impostato in batch size)
    a. Il modello predice un punteggio per ciascuna immagine.
    b. Viene calcolato l'errore rispetto al punteggio corretto (MES loss).
    c. L'errore viene propagato all'indietro nella rete (backpropagation) per calcolare quanto ogni peso ha contribuito all'errore.
2. **Validazione**: le immagini del set di validazione (mai viste durante il training) vengono valutate per misurare la performance reale.
3. **Salvataggio**: se la loss di validazione è la migliore ottenuta finora, il modello viene salvato sul disco.

Di seguito viene riportata la stampa di una esecuzione di training.

```
Loaded 614 samples
Using device: cpu
Using cache found in C:\Users\mchiavaroli/.cache\torch\hub\pytorch_vision_main
✅ Loaded pretrained ImageNet weights for backbone
Epoch [1/20] Train Loss: 0.0394 | Val Loss: 0.0260 | ~Mean Absolute Error: 16.1
  → Saved best model (val_loss=0.0260)
Epoch [2/20] Train Loss: 0.0159 | Val Loss: 0.0135 | ~Mean Absolute Error: 11.6
  → Saved best model (val_loss=0.0135)
Epoch [3/20] Train Loss: 0.0093 | Val Loss: 0.0121 | ~Mean Absolute Error: 11.0
  → Saved best model (val_loss=0.0121)
Epoch [4/20] Train Loss: 0.0055 | Val Loss: 0.0094 | ~Mean Absolute Error: 9.7
  → Saved best model (val_loss=0.0094)
Epoch [5/20] Train Loss: 0.0059 | Val Loss: 0.0099 | ~Mean Absolute Error: 10.0
Epoch [6/20] Train Loss: 0.0063 | Val Loss: 0.0088 | ~Mean Absolute Error: 9.4
  → Saved best model (val_loss=0.0088)
Epoch [7/20] Train Loss: 0.0059 | Val Loss: 0.0064 | ~Mean Absolute Error: 8.0
  → Saved best model (val_loss=0.0064)
Epoch [8/20] Train Loss: 0.0040 | Val Loss: 0.0059 | ~Mean Absolute Error: 7.7
  → Saved best model (val_loss=0.0059)
```

Dalla stampa si può notare come ad ogni epoca, il modello tende a migliorare la fase di validazione, portando così il valore dell'errore sempre più verso lo zero.

Fase 3 — Servizio di Inferenza (main.py)

Il modello addestrato viene esposto come servizio REST tramite FastAPI. All'avvio, il server carica i pesi da weights/cleanliness_model.pth. Se il file non esiste (modello non ancora addestrato), il servizio restituisce punteggi casuali qualora il modello non sia stato precedentemente addestrato:

L'endpoint principale è:

```
POST /predict
Input:  { "image_urls": ["url1", "url2", ...] }
Output: { "scores": [82.5, 45.3], "average": 63.9, "model_loaded": true }
```

Per ogni URL ricevuta, il servizio scarica l'immagine tramite httpx (client HTTP asincrono), applica il preprocessing standard (resize 224×224 + normalizzazione ImageNet) e produce il punteggio tramite forward pass. Il risultato è una lista di punteggi individuali e la loro media.

Il middleware CORS è configurato per accettare richieste dal frontend Next.js (localhost:3000 in sviluppo); la variabile d'ambiente PYTHON_API_URL consente di configurare l'URL del servizio ML in produzione.

2.6 Stack Tecnologico Completo

Tecnologia	Ruolo nel progetto
Next.js 16	Framework React full-stack con App Router e API Routes integrate.
React 19	Libreria UI con support a Server e Client Components.
TypeScript 5.7	Type safety sull'intero codebase frontend.
Tailwind CSS 4	Framework CSS utility-first per styling responsivo.
shadcn/ui + Radix UI	Componenti UI accessibili e personalizzabili.
SWR	Libreria di data fetching con caching e revalidazione.
Supabase	BaaS (PostgreSQL + Storage + Auth) per persistenza e file storage.
FastAPI	Framework Python per API REST ad alte prestazioni (ASGI/uvicorn).
PyTorch + torchvision	Framework deep learning per training e inferenza.
EfficientNet-B0	Architettura CNN pre-addestrata su ImageNet (transfer learning).
httpx	Client HTTP asincrono per il download delle immagini nel servizio ML.
Pillow (PIL)	Libreria Python per manipolazione e preprocessing immagini.
pnpm	Package manager veloce per il progetto Node.js.

2.7 Struttura del Repository

```
Challenge-AI/
├── app/                # Next.js App Router
│   ├── api/            # API Routes (BFF)
│   │   ├── palazzi/    # CRUD palazzi + endpoint cleanliness
│   │   ├── vetrate/    # CRUD vetrate
│   │   └── upload/     # Upload immagini su Supabase Storage
│   ├── page.tsx        # Pagina principale
│   └── layout.tsx      # Layout globale
├── components/         # Componenti React riutilizzabili
├── lib/                # Tipi TypeScript e client Supabase
├── python/             # Servizio ML Python
│   ├── label_images.py # Etichettatura automatica immagini
│   ├── train.py        # Script di addestramento
│   ├── model.py        # Architettura modello + funzione predict
│   ├── main.py         # Server FastAPI
│   ├── dataset/        # Dataset immagini + labels.csv
│   └── weights/        # Pesi del modello addestrato (.pth)
├── scripts/            # Script SQL (creazione tabelle)
└── public/             # Asset statici
```

2.8 Flusso Dati End-to-End

Il flusso completo per il calcolo del livello di pulizia è il seguente:

Step	Descrizione
1	L'utente apre la pagina di dettaglio di un palazzo e clicca "Calcola livello pulizia".
2	Il componente React esegue POST /api/palazzi/[id]/cleanliness.
3	L'API Route Next.js interroga Supabase e recupera le image_url di tutte le vetrate del palazzo.
4	L'API Route chiama POST http://[PYTHON_API_URL]/predict con la lista di URL.
5	Il servizio FastAPI scarica ogni immagine (httpx), esegue resize+normalizzazione (PIL/torchvision) e forward pass (EfficientNet-B0).
6	FastAPI restituisce { scores, average, model_loaded }.
7	L'API Route Next.js inoltra la risposta al client React.
8	Il componente visualizza il punteggio con colore e label qualitativa (Critico/Buono/Eccellente).

2.9 Limiti e assunzioni

Nel progettare e sviluppare il sistema descritto, sono state adottate alcune assunzioni semplificative e sono presenti determinati limiti tecnici e metodologici che è importante esplicitare.

Assunzioni

- Qualità delle immagini: si assume che le immagini caricate dagli utenti siano sufficientemente nitide, ben illuminate e rappresentative dello stato reale delle vetrate. Il modello non è progettato per gestire immagini fortemente degradate o fuori fuoco.

Limiti

- **Dataset limitato**: la quantità e varietà del dataset possono influire sulla capacità di generalizzazione del modello, aumentando il rischio di overfitting o bias.
- **Dipendenza da servizi esterni**: il corretto funzionamento del sistema dipende dalla disponibilità del servizio di inferenza ML e, opzionalmente, dello storage cloud.
- **Interpretabilità limitata**: il modello di deep learning fornisce un punteggio numerico senza spiegazioni dettagliate delle decisioni (black-box), rendendo difficile interpretare i motivi specifici di una valutazione.